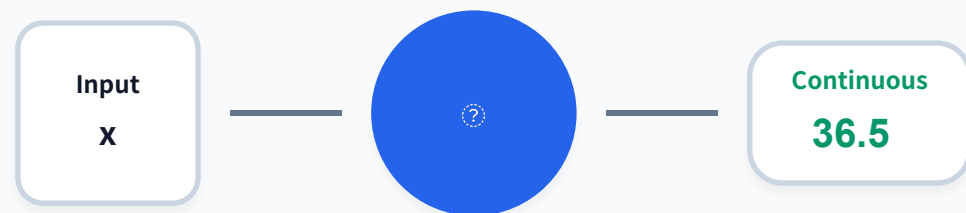


컴퓨터비전 _인공지능 (AI)개론

7. 선형 회귀

- 문제의 정의
- 선형 함수(`nn.Linear`)
- 커스텀 클래스를 이용한 모델 정의
- `MSELoss` 클래스를 이용한 손실 함수
- 데이터 준비
- 모델 정의
- 경사 하강법
- 결과 확인
- 중회귀 모델로 확장
- 학습률의 변경

회귀 (REGRESSION)



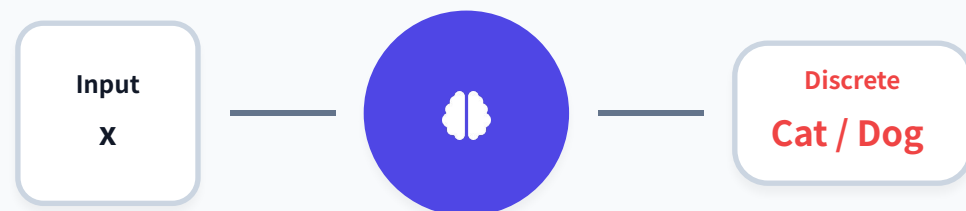
예측값이 연속된 실수 (Continuous Value)

집값 예측

온도

판매량

분류 (CLASSIFICATION)



예측값이 이산적 클래스 (Discrete Label)

스팸 필터

암 진단

품종 분류



회귀 (Regression)

출력값이 연속적인 수치(Float)인 문제.
입력값(x)에 대응하는 정확한 수치(y)를
근사(Approximation)하는 것이 목표.



분류 (Classification)

출력값이 정해진 카테고리(Label) 중 하나.
데이터를 구분하는 결정 경계
(Decision Boundary)를 찾는 것이 핵심.



핵심 차이

"얼마나 많이?" (How much)
vs "무엇인가?" (Which one)

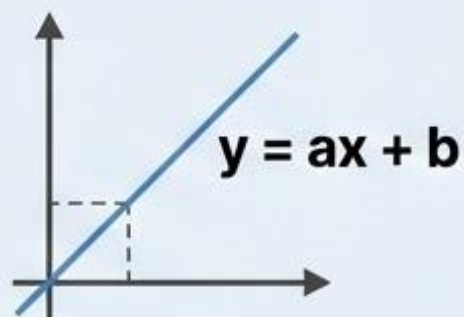
딥러닝 모델을 거대한 레고 성이라고 상상해 보세요.

화려한 탑과 튼튼한 성벽 모두, 결국 가장 기본이 되는 '2x4' 레고 블록으로 만들어집니다.

딥러닝 세계에서 바로 그 기본 블록 역할을 하는 것이 `nn.Linear`, 즉 선형 함수입니다.

이 간단한 함수는 입력된 정보의 각 부분에 '중요도(가중치)'를 매기고, 기본 '보정값(편향)'을 더해 새로운 정보를 만들어냅니다.

수학적 정의



1차 함수 $y=ax+b$ 를 텐서의 세계로 확장한 것입니다. 입력과 출력이 직선적인 관계를 갖는 함수를 의미합니다.

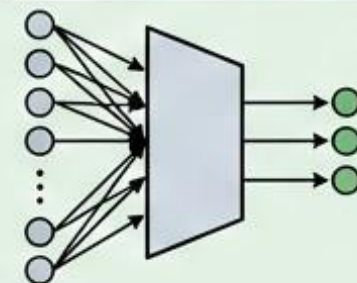
레시피 조절기



커피를 만들 때 원두의 양에 따라 진하게 추출할지를 정하고, 기본 설량의 양을 더해 최종 맛을 결정하는 것과 같습니다.

핵심 인자

```
nn.Linear(10, 3)
```



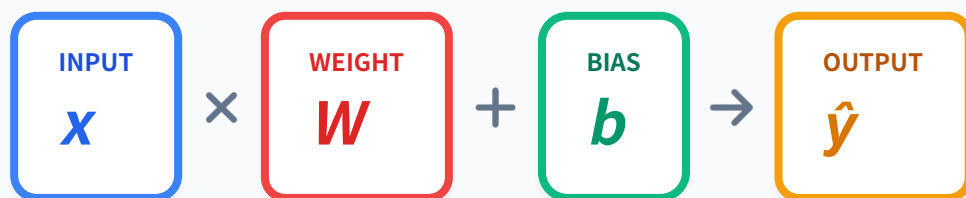
`in_features`(입력 특성 수)와 `out_features`(출력 특성 수) 두 가지 핵심 인자로 생성됩니다. 예: 10개 입력을 3개 출력으로 변환

√ 핵심 수식

$$y = Wx + b$$

입력과 가중치의 선형 결합 + 편향

구조 시각화

스칼라: $y = w_1x_1 + \dots + b$ 행렬: $Y = XW + b$ 

학습 파라미터

W (Weight): 영향력의 크기 (기울기)

b (Bias): 기본 출력값 (절편)



차원 (Dimensions)

입력이 벡터면 가중치도 벡터입니다.

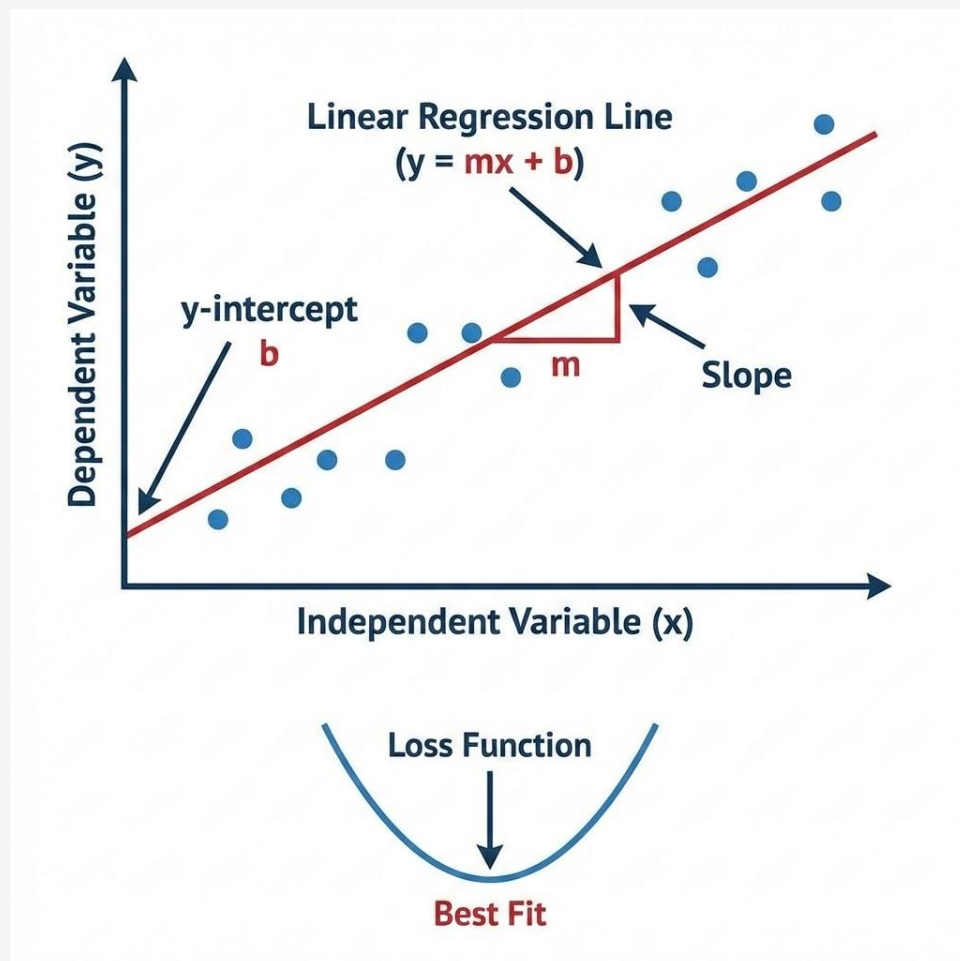
 $x \in \mathbb{R}^d$, $W \in \mathbb{R}^{1 \times d}$, $b \in \mathbb{R}$ 

모델의 가정

입력 x와 출력 y 사이의 선형(Linear) 관계를 가정

오차의 제곱합(Sum of Squared Errors)을 최소화

LEAST SQUARES VISUALIZATION



목적: 오차 제곱합의 최소화

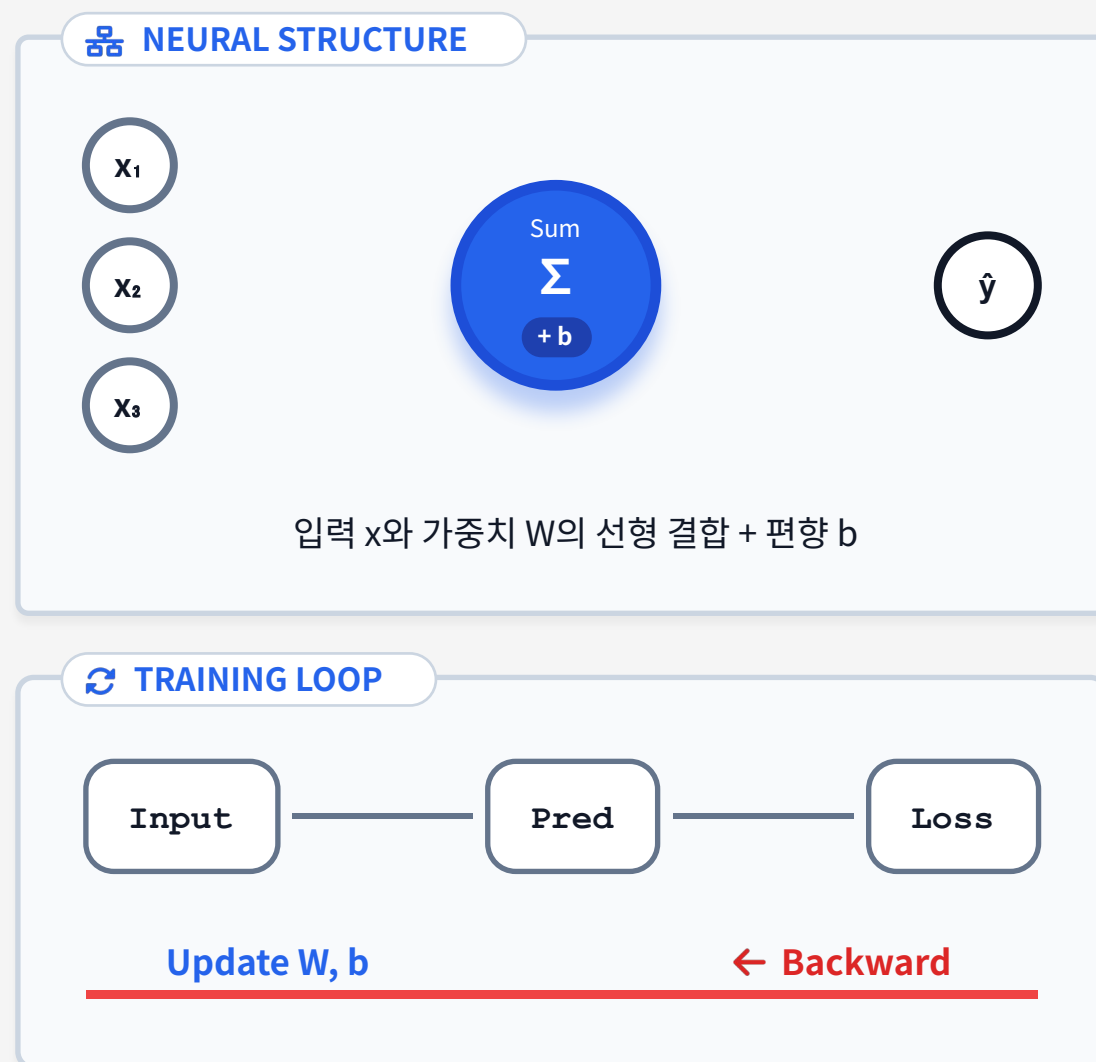
모든 데이터 포인트에 대해
오차(잔차) = 실제값 - 예측값
이들의 제곱의 합(Sum of Squares)을
가장 작게 만드는 선을 찾습니다.



제곱의 효과

오차를 제곱하기 때문에,
큰 오차(이상치)에 대해서는
훨씬 더 큰 벌점(Penalty)을 부여합니다.

선형회귀 = 뉴런 1개짜리 모델



단일 뉴런 (Single Neuron)

선형회귀는 활성화 함수가 없는
1개의 뉴런과 수학적으로 동일합니다.

연산 그래프
(Computational Graph)

모델 연산: $z = Wx + b$
손실 계산: $L = \text{MSE}(y, z)$



학습 원리: 역전파 (Backprop)

Loss를 줄이는 방향으로 Gradient를
계산해 파라미터 W, b 를 스스로
갱신합니다.

선형 변환 $y = xW^T + b$ 를 수행하는 핵심 모듈



모듈 구조

nn.Module을 상속받아 구현
입력 벡터에 선형 변환을 적용하여
출력 벡터 생성



학습 파라미터

.weight: 가중치 ($\text{out} \times \text{in}$)
.bias: 편향 (out) 자동 미분 가능
(requires_grad=True)



입출력 차원

생성 시 in_features와 out_features 지정
필수 데이터 특징(Feature) 수와 일치

기본 생성 방법

```
l = nn.Linear(2, 3)
```

입력이 2차원, 출력이 3차원인 선형 함수를 정의합니다.

실행 결과

```
Linear(in_features=2, out_features=3, bias=True)
```

in_features와 out_features가 지정한 값으로 설정되고, bias는 기본적으로 True로 설정됩니다.

1입력 1출력 선형 함수

01

함수 생성

```
torch.manual_seed(123)
l1 = nn.Linear(1, 1)
```

가장 기본적인 $y=ax+b$ 형태의 회귀 모델에 해당합니다.

02

파라미터 확인

```
weight: tensor([[ -0.4078]], requires_grad=True)
bias: tensor([ 0.0331], requires_grad=True)
```

두 파라미터 모두 `requires_grad=True`로 설정되어 학습 대상임을 나타냅니다.

03

초깃값 설정

```
nn.init.constant_(l1.weight, 2.0)
nn.init.constant_(l1.bias, 1.0)
```

$y=2x+1$ 함수를 직접 만들어 테스트할 수 있습니다.

입력 데이터

```
# -2부터 2까지 1씩 증가하는 NumPy 배열 생성
x_np = np.arange(-2.0, 2.1, 1.0)

# NumPy 배열을 PyTorch 텐서로 변환
x = torch.tensor(x_np).float()
# .float()으로 데이터 타입을 float32로 지정

# (N, 1) 사이즈로 변경. N은 데이터 개수.
# 선형 함수는 기본적으로 배치(묶음) 데이터를
# 처리하므로 2차원 텐서 입력을 가정합니다.
x = x.view(-1, 1)

# 11 함수에 x를 입력하여 y를 계산합니다.
y = l(x)
```

출력 결과

```
y의 data:
tensor([[ -3.],
         [-1.],
         [ 1.],
         [ 3.],
         [ 5.]])
```

$y=2x+1$ 수식에 따라 정확하게 계산되었습니다.

입력 $[-2, -1, 0, 1, 2]$ 에 대해 $y=2x+1$ 을 계산한 결과인 $[-3, -1, 1, 3, 5]$ 가 정확하게 출력됨

1

모델 정의

```
# 입력: 2, 출력: 1 선형 함수 정의
l2 = nn.Linear(2, 1)

# 초깃값 설정
# 모든 가중치는 1.0,
# 편향은 2.0으로 설정
nn.init.constant_(l2.weight, 1.0)
nn.init.constant_(l2.bias, 2.0)
```

$y=x_1+x_2+2$ 함수를 생성합니다.

2

테스트 데이터

```
# 테스트용 2차원 데이터 준비
x2_np = np.array([[0, 0], [0, 1],
                  [1, 0], [1, 1]])
x2 = torch.tensor(x2_np).float()
```

4가지 입력 조합을 테스트합니다.

3

결과 확인

```
y2의 data:
tensor([[2.],
        [3.],
        [3.],
        [4.]])
```

수식에 맞게 정확히 계산되었습니다.

입력은 2개, 출력은 3개인 경우를 살펴보겠습니다.

이는 여러 개의 출력을 동시에 예측해야 할 때 사용됩니다.

가중치 설정

```
# 입력: 2, 출력: 3 선형 함수 정의
l3 = nn.Linear(2, 3)

# 초깃값 설정 (weight 행렬의 각 행을
# 다른 값으로 설정)
nn.init.constant_(l3.weight[0, :],
1.0)
nn.init.constant_(l3.weight[1, :],
2.0)
nn.init.constant_(l3.weight[2, :],
3.0)
nn.init.constant_(l3.bias, 2.0)
```

수학적 표현

$$y_1 = 1 \cdot x_1 + 1 \cdot x_2 + 2$$

$$y_2 = 2 \cdot x_1 + 2 \cdot x_2 + 2$$

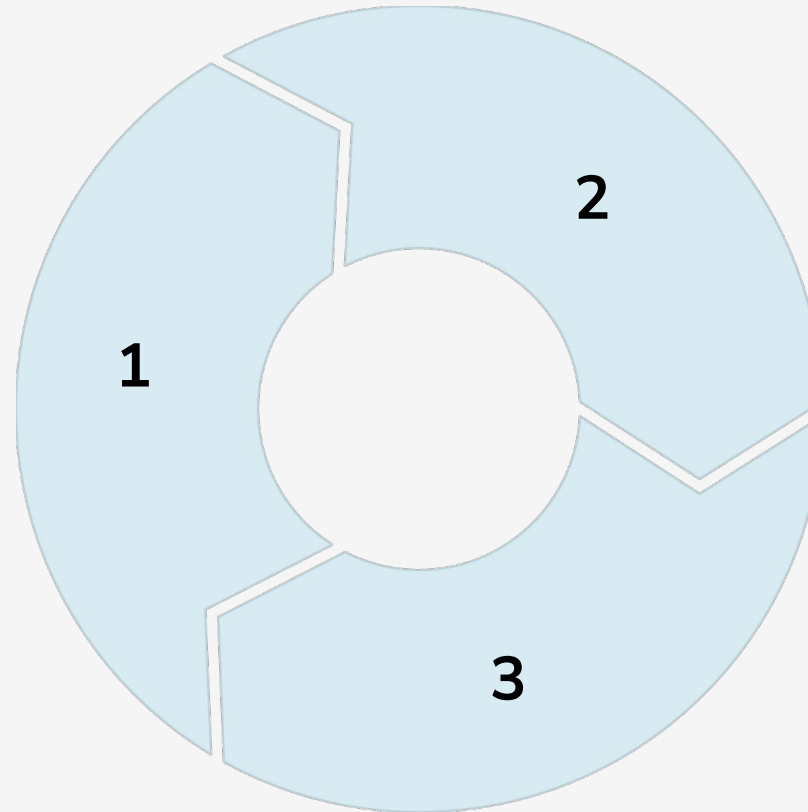
$$y_3 = 3 \cdot x_1 + 3 \cdot x_2 + 2$$

출력 결과

```
y3의 data:
tensor([[2., 2., 2.],
        [3., 4., 5.],
        [3., 4., 5.],
        [4., 6., 8.]])
```

여러 블록을 체계적으로 조립하고, 이 조립품에 이름을 붙여주고 싶을 때 파이썬 클래스를 사용합니다.

예측 함수 (net)
입력 데이터를 받아 예측값을
만들어내는 모델의 본체입니다.



손실 함수 (criterion)

모델의 예측값과 실제 정답을 비교하여
'오차'를 계산합니다.

최적화 함수 (optimizer)

계산된 오차를 바탕으로 모델의
파라미터를 어느 방향으로 얼마나
수정할지 결정하고 실행합니다.

__init__ 메서드

모델의 '뼈대'를 구성하는 곳입니다.

모델이 사용할 신경망 레이어들을 '선언'하고 초기화합니다.

건물이 지어질 때 어떤 재료(레이어)들이 필요한지 목록을 작성하는 단계와 같습니다.

forward 메서드

데이터가 모델을 통과하는 '흐름'을 정의하는 곳입니다.

입력 데이터가 어떤 순서로, 어떤 계산을 거쳐 최종 예측값으로 변환되는지를 구체적으로 코딩합니다.

건물의 설계도에서 각 방과 복도가 어떻게 연결되어 있는지를 그리는 것과 같습니다.

```
class Net(nn.Module): # 모든 모델의 부모 클래스인 nn.Module을 상속받습니다.
    def __init__(self, n_input, n_output):
        super().__init__()

        # 출력층 정의: 입력 특성 수(n_input)와 출력 특성 수(n_output)를 받는 선형 레이어(l1)를 정의합니다.
        self.l1 = nn.Linear(n_input, n_output)

    # 예측 함수 정의: 데이터의 순전파 흐름을 정의합니다.
    def forward(self, x):
        x1 = self.l1(x)
        return x1
```

1 클래스 상속

nn.Module을 상속받아 파이토치의 강력한 기능들을 모두 물려받습니다.

2 부모 클래스

추가 `super().__init__()`로 모델이 정상적으로 동작하는 데 필요한 기본 설정을 수행합니다.

3 순전파 정의

forward 메서드에서 데이터의 흐름을 정의합니다.

인스턴스 생성

```
# 더미 입력 데이터 생성 (100개의 샘플, 1개의 특성)
inputs = torch.ones(100, 1)

# 인스턴스 생성 (1 입력, 1 출력 선형 모델)
n_input = 1
n_output = 1
net = Net(n_input, n_output)

# Net 클래스의 인스턴스를 생성합니다.

# 예측 수행
outputs = net(inputs)
```

실행 결과

```
torch.Size([100, 1])
tensor([[0.6176],
        [0.6176],
        [0.6176],
        [0.6176],
        [0.6176]], grad_fn=<SliceBackward0>)
```

net 인스턴스를 마치 함수처럼 호출하면, 파이토치가 내부적으로 net.forward(inputs)를 실행하여 예측을 수행합니다.

손실 함수: MSE (Mean Squared Error)

회귀 문제의 표준 손실 함수 $\text{Loss} = \text{mean}((y - \hat{y})^2)$



정의 및 의미

예측값(pred)과 실제값(target) 차이의 제곱 평균.
학습 중 모델의 "틀린 정도"를 수치화.



특징: 패널티

미분 가능하여 경사 하강법 적용 용이.
오차를 제곱하므로 큰 오차에 더 큰 벌점 부여.



주의: 스케일 민감성

타겟 변수(y)단위에 따라 손실 크기가 달라짐.
데이터 표준화(Normalization) 권장.

```
mse_loss_example.py

import torch
import torch.nn as nn

# 1. MSE 손실 함수 정의
loss_fn = nn.MSELoss(reduction='mean')

# 2. 데이터 준비 & 손실 계산
pred = torch.tensor([2.5, 0.0, 1.8], requires_grad=True)
target = torch.tensor([3.0, -0.5, 2.0])
loss = loss_fn(pred, target)
print(f"MSE Loss: {loss.item():.4f}") # Output: 0.1767
```

손실 함수 정의

```
criterion = nn.MSELoss()  
labels1 = torch.zeros(100, 1)
```

평균 제곱 오차를 계산하는
손실 함수를 생성합니다.

손실 계산

```
loss =\  
criterion(outputs, labels1) / 2.0
```

예측값과 정답 사이의
오차를 계산합니다.

경사 계산

```
loss.backward()
```

loss.backward()로 모든 파라미터의
경사를 자동으로 계산합니다.

모델을 학습시킬 데이터를 준비합니다.

보스턴 주택 가격 데이터셋을 사용하여 '방의 평균 개수'로 '주택 가격'을 예측하는 간단한 문제를 풀어보겠습니다.

506

데이터 개수

총 506개의 주택 데이터가 있습니다.

13

특성 개수

각 데이터는 13개의 특성을 가집니다.

1

사용 특성

이 중 'RM'(방의 개수) 하나만 사용합니다.

방 개수와 가격의 관계를 산포도로 확인해보겠습니다.

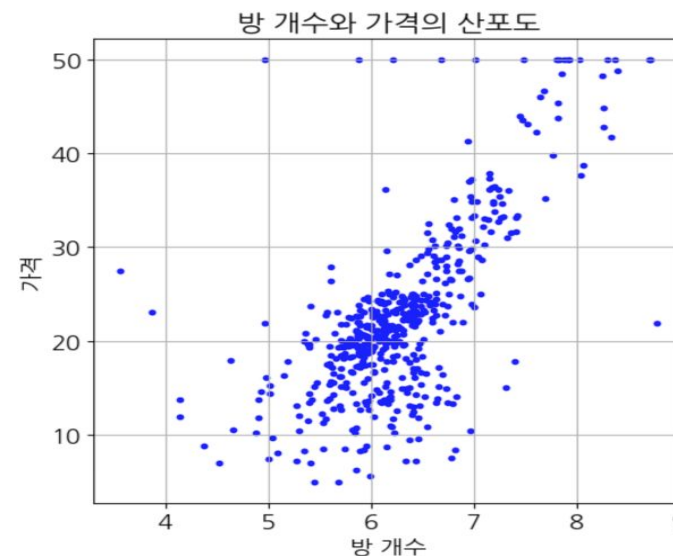
전반적으로 '방 개수'가 많아질수록 '가격'이 높아지는 양의 선형 관계가 있음을 짐작할 수 있습니다.

입력 데이터 예시

```
raw_df = pd.read_csv(data_url, sep="\s+",
                     skiprows=22, header=None)
x_org = np.hstack([raw_df.values[:,2, :],
                   raw_df.values[1::2, :2]])
yt = raw_df.values[1::2, 2]
feature_names = np.array(['CRIM', 'ZN', 'INDUS',
                          'CHAS', 'NOX', 'RM', 'AGE',
                          'DIS', 'RAD',
                          'TAX', 'PTRATIO', 'B',
                          'LSTAT'])

x = x_org[:, feature_names == 'RM'] # RM 특성만 저장
```

산포도 출력



우리가 만들 모델은 이 점들의 분포를 가장 잘 대표하는 하나의 직선을 찾는 것을 목표로 합니다.

```
class Net(nn.Module):
    def __init__(self, n_input, n_output):
        super().__init__()
        # 출력층 정의: 입력 차원에서 출력 차원으로 가는 선형 변환
        self.l1 = nn.Linear(n_input, n_output)

        # 초깃값을 모두 1로 설정
        nn.init.constant_(self.l1.weight, 1.0)
        nn.init.constant_(self.l1.bias, 1.0)

    # 예측 함수 정의
    def forward(self, x): # 데이터가 모델을 통과하는 흐름을 정의
        x1 = self.l1(x) # 선형 회귀 계산
        return x1
```

1개의 입력을 받아 1개의 출력을 내보내는 선형 회귀 모델을 정의했습니다.

손실 함수

```
criterion = nn.MSELoss()
```

평균 제곱 오차를 계산하는 역할을 합니다.

예측값과 정답 사이의 차이를 제공하여 평균을 냅니다.

최적화 함수

```
lr = 0.01  
optimizer = optim.SGD(net.parameters(), lr=lr)
```

경사 하강법을 수행합니다.

학습률 0.01로 파라미터를 업데이트할 보폭을 결정합니다.

01

예측 계산

```
outputs = net(inputs)
```

현재 파라미터를 사용해 입력 데이터에 대한 예측값을 계산합니다.

02

손실 계산

```
loss = criterion(outputs, labels1)
```

예측값과 실제 정답을 손실 함수에 넣어 오차를 계산합니다.

03

경사 계산

```
loss.backward()
```

손실을 줄이기 위해 각 파라미터를 어느 방향으로 움직여야 할지 계산합니다.

04

파라미터 수정

```
optimizer.step()  
optimizer.zero_grad()
```

계산된 기울기를 바탕으로 파라미터를 실제로 수정하고 경사값을 초기화합니다.

```
# 학습률
lr = 0.01

# 인스턴스 생성 (학습 시작 전 파라미터 값 초기화)
net = Net(n_input, n_output)

criterion = nn.MSELoss() # 손실 함수: 평균 제곱 오차

optimizer = optim.SGD(net.parameters(), lr=lr) # 최적화 함수: 경사 하강법

num_epochs = 5000 # 반복 횟수

# 평가 결과 기록 (손실 값만 기록)
history = np.zeros((0, 2))

# 반복 계산 메인 루프
for epoch in range(num_epochs):
    optimizer.zero_grad() # ① 경사값 초기화 (가장 먼저 수행!)

    outputs = net(inputs) # ② 예측 계산

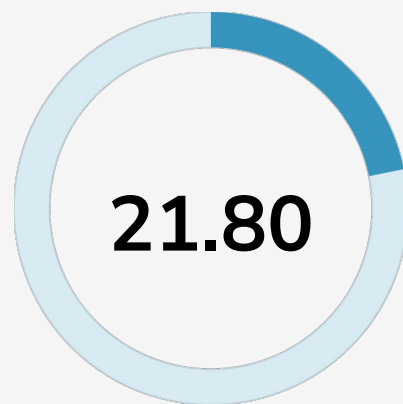
    loss = criterion(outputs, labels1) / 2.0 # ③ 손실 계산

    loss.backward() # ④ 경사 계산

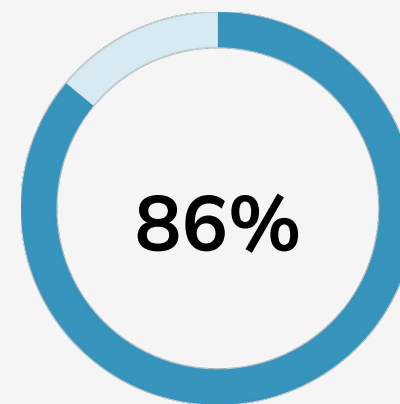
    optimizer.step() # ⑤ 파라미터 수정
```



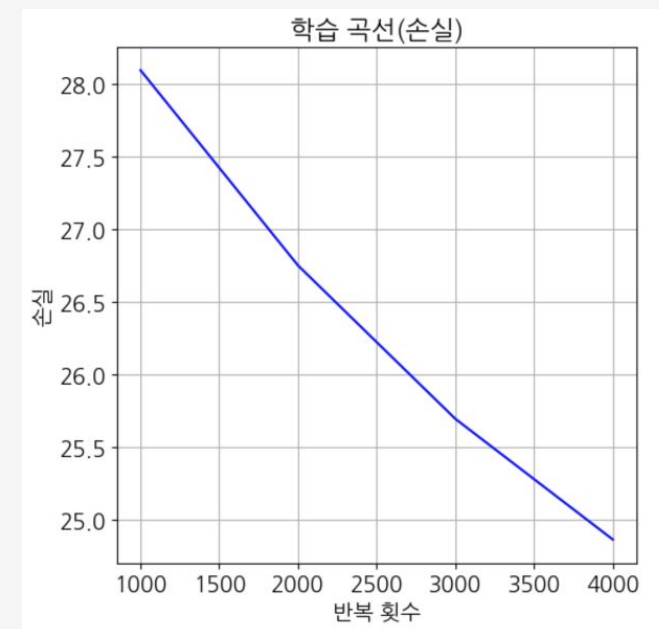
초기 손실값
학습 시작 시점의 오차입니다.



최종 손실값
5000번 학습 후의 오차입니다.



개선율
손실이 크게 감소하여 모델이
성공적으로 학습되었습니다.



학습 곡선을 보면 예쁜 우하향 곡선을 얻게 되어, 학습이 문제없이 이뤄졌음을 알 수 있습니다.
회귀 직선도 산포도상의 학습 데이터와 비교해 알맞게 근사되었습니다.

2입력 모델 확장

RM(방 개수)과 LSTAT(저소득자 비율) 두 특성을 사용하여 모델을 확장했습니다.

파라미터 수가 3개(weight 2개, bias 1개)로 증가했습니다.

학습률 0.001로 해결

학습률을 1/10로 줄이니 최종 손실값이 15.28로 안정적으로 수렴했습니다.

1입력 모델(21.8)보다도 더 나은 성능을 보였습니다.

1

2

학습률 0.01의 문제

과거 학습률로 학습하니 손실값이 $\text{inf} \rightarrow \text{nan}$ 으로 발산했습니다.
학습률이 너무 커서 정답 근처를 지나쳐 버리는 현상이 발생함

3

머신러닝에서 학습률이 매우 중요한 하이퍼파라미터라는 것을 확인했습니다.

적절한 학습률 설정이 성공적인 학습의 핵심입니다.

파이토치에서 모델 인스턴스(net)를 함수처럼 net(inputs)로 호출하는 것은 nn.Module의 내장 기능 덕분입니다.

우리가 스마트폰 앱 아이콘을 터치하듯이

net(__call__), net(inputs)는 복잡한 전처리 후 우리가 정의한 기능(forward)을 실행합니다.



01

net(inputs) 호출

모델 인스턴스를 함수처럼 사용합니다.

02

nn.Module의 __call__ 호출

부모 클래스에 정의된 __call__ 함수가 자동으로 실행됩니다.



03

정의된 forward 호출

__call__ 내부에서 우리가 작성한 forward 함수가 안전하게 호출됩니다.

이러한 연쇄적인 구조를 통해 파이토치는 모델의 핵심 로직(forward) 외에 메모리 관리, Hook 처리 등 다양한 내부 작업을 자동으로 수행합니다.

파이토치의 '바이어스'는 수학적 관점에서 '더미 변수에 해당하는 가중치'와 같습니다.

□ 주소 표기 비유:

- '서울특별시 강남구 테헤란로 101'과 '서울특별시 강남구 역삼동 821'은 다르지만 같은 장소를 가리킵니다.
- 바이어스 항을 별도로 두는 방식과 더미 변수를 사용하는 방식은 표기법의 차이일 뿐, 근본적으로 동일한 계산을 수행합니다.

두 방식의 수학적 표현 비교:



- 파이토치 방식: $y = wx + b$ (가중치 w 와 바이어스 b 를 별개로 취급)
- 더미 변수 방식: $y = (w_1, w_0) \cdot (x, 1)$ (바이어스 b 를 더미 변수 1에 곱해지는 가중치 w_0 으로 취급)

파이토치가 바이어스를 별도로 처리하는 이유는

사용자가 입력 데이터 x 에 매년 1이라는 더미 변수 열을 추가해야 하는 번거로움을 덜어주기 위함입니다.

R² Score: 정의와 수식

모델이 데이터의 분산을 얼마나 설명하는지 나타내는 결정 계수 (Coefficient of Determination)



설명력 (Explanatory Power)

전체 데이터 변동(분산) 중 모델이
예측으로 설명한 비율.
1.0에 가까울수록 완벽.



상대적 성능 기준

단순 평균 모델($R^2=0$) 대비 개선도 측정.
음수 = 평균보다 못한 모델.

pytorch_r2_score.py

```
def r2_score(y_true, y_pred):  
    # SS_tot: 타겟 데이터의 총 분산 (평균 기준 변동량)  
    y_mean = y_true.mean()  
    ss_tot = ((y_true - y_mean) ** 2).sum()  
    # SS_res: 잔차 제곱합 (Residual Sum of Squares)  
    ss_res = ((y_true - y_pred) ** 2).sum()  
    return 1 - (ss_res / ss_tot)
```




$$-\infty < R^2 \leq 1$$

**"1에 가까울수록 데이터의
변동성을 완벽히 설명한다"**

단순 평균(Baseline) 모델 대비
성능이 얼마나 개선되었는가?



R² ≈ 1 (Best)

모델이 데이터의 분산(Variance)을 대부분 설명함.
예측값($\hat{y}$)이 실제값(y)과 매우 유사.



R² ≈ 0 (Baseline)

모델이 단순히 타겟의 평균값을 출력함.
입력(x)이 예측에 기여하지 못하는 상태.



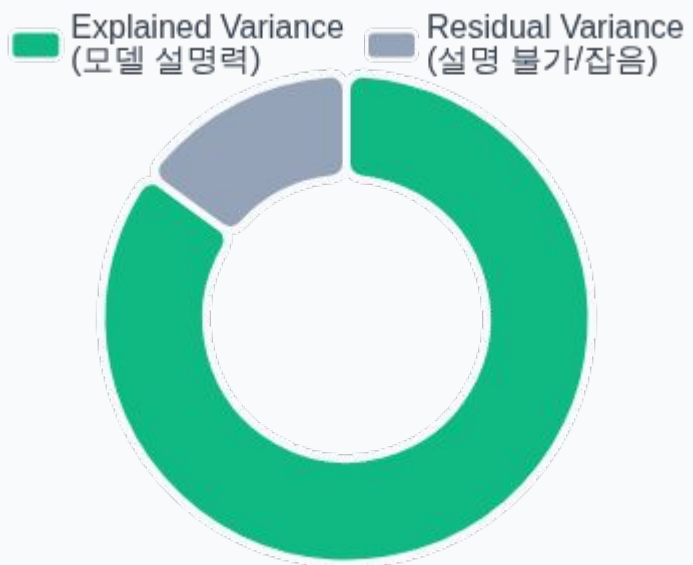
R² < 0 (Warning)

평균만도 못한 예측 (예: 데이터 역행).
모델 구조 오류 혹은 극심한 과적합 의심.

R² Score 값 85% 의미

총 변동(Total Variance)의 분해와 해석

VARIANCE BREAKDOWN



$$\text{Total Variance} = \text{Explained} + \text{Residual}$$

전체 데이터의 움직임 = 모델이 설명한 것 + 설명 못 한 잡음



"85%를 설명한다"

데이터가 가진 전체 변동성 중 85%는 모델(입력 변수)로 예측 가능하다는 의미입니다. 나머지 15%는 설명할 수 없는 오차(Noise)입니다.



트렌드 vs 잡음 분해

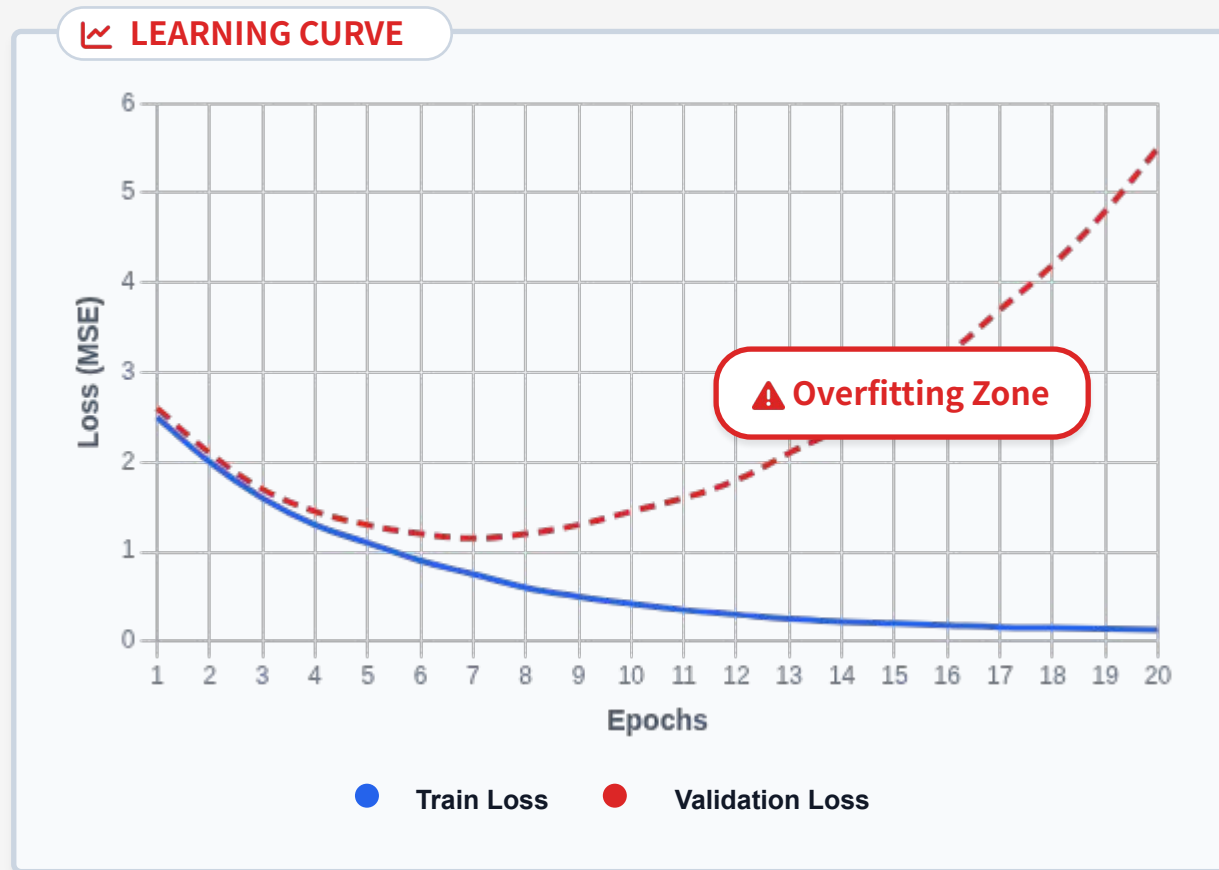
모델은 데이터의 주요 경향성(Trend)을 잡아내고, R²는 그 경향성이 전체 데이터 분포에서 얼마나 큰 비중을 차지하는지 보여줍니다.



주의: 인과관계가 아님

높은 R²가 반드시 원인과 결과를 증명하진 않습니다. 단순히 데이터 패턴이 선형식과 잘 맞아떨어진다는 것을 의미할 뿐입니다.

훈련 데이터에는 완벽하지만 실전에는 약한 모델



오버피팅이란

?

모델이 훈련 데이터를 과도하게 암기하여, 새로운 데이터(검증셋)에 대한 일반화 성능이 떨어지는 현상.

주요 징후 (Symptoms)

Train Loss는 계속 줄어드는데, Validation Loss는 오히려 증가하며 격차가 벌어짐.

해결 방안

1. 정규화 (Ridge, Lasso)
2. 데이터 추가 (Augmentation)
3. 조기 종료 (Early Stopping)

가중치의 제곱합을 패널티로 부과하여 과적합을 억제하는 정규화 기법



정의 및 수식

손실 함수에 L2 패널티 추가
 $\text{Loss} = \text{MSE} + \lambda \|W\|^2$



직관적 의미

"모든 특성을 조금씩 사용하자"

- 계수를 0으로 만들지 않음
- 잡음(Noise) 민감도 감소



언제 사용하는가

- 다중공선성 문제 해결
- 안정적인 예측 필요 시
- 특성 제거 없이 규제할 때



pytorch_ridge_decay.py

```
import torch.optim as optim

# Ridge(L2)는 weight_decay 파라미터로 구현 (λ 역할)
lambda_12 = 1e-2 # 규제 강도 설정

optimizer = optim.SGD(
    model.parameters(),
    lr=0.1,
    weight_decay=lambda_12) # 핵심: 가중치 감소(L2 Regularization)
```

가중치 절대값 합을 패널티로 사용하여 희소성(Sparsity)을 유도



정의 및 수식

$\text{Loss} = \text{MSE} + \lambda \sum |W|$
오차와 가중치 절대값의 합을 동시에 최소화.



직관: Feature Selection

덜 중요한 특성의 가중치를 완전히 0으로 만들.
자동으로 변수를 선택하는 효과.



언제 사용하는가?

입력 변수(Feature)가 너무 많거나,
모델의 해석력이 중요할 때.

lasso_implementation.py

```
# PyTorch Optimizer는 L1을 직접 지원하지 않아 Loss에 추가 필요
def training_step(model, x, y, lambda_l1=0.01):
    pred = model(x); mse_loss = mse_fn(pred, y) # 1. MSE 계산
    # 2. 모든 가중치(Bias 제외)의 절대값 합 계산
    l1_norm = sum(p.abs().sum() for n, p in model.named_parameters() if 'weight' in n)
    # 3. 최종 손실 함수 정의 (MSE + L1 Penalty)
    loss = mse_loss + lambda_l1 * l1_norm
    loss.backward(); optimizer.step()
```

**Ridge (L2)****가중치 제곱의 합**

$$\text{Penalty} = \lambda \|W\|^2$$

0에 가깝게 축소

완전히 0이 되지는 않음 (모든 변수 활용)

안정적 (Stable)

데이터 변동에 덜 민감함

다중공선성 해결 시

변수 간 상관관계가 높을 때 유리



규제 방식



계수 특성



안정성



사용 시점

**Lasso (L1)****가중치 절대값의 합**

$$\text{Penalty} = \alpha \|W\|_1$$

0으로 수렴 가능

불필요한 변수 제거 (Sparse Model)

상대적 불안정

상관관계 높은 변수 중 하나만 임의 선택

Feature Selection 필요 시

해석력이 중요한 고차원 데이터

Ridge와 Lasso의 장점을 모두 취하는 하이브리드 정규화 기법



결합된 규제

손실 함수에 L1과 L2 패널티를 동시에 적용.
 $\text{Loss} = \text{MSE} + \alpha|W|_1 + \lambda|W|^2$



희소성 & 안정성

Lasso의 변수 선택 기능(0으로 만들)과 Ridge의 다중공선성 해결(분산 감소)을 동시에 달성하여 균형 잡힘.



실무 활용 최적

특성이 많고 상관관계가 높은 실제 데이터셋에서 단일 규제보다 우수. 1순위 고려 모델.

elastic_net_training.py

```
# 1. L2 규제는 Optimizer의 weight_decay로 적용
optimizer = torch.optim.SGD(model.parameters(), lr=0.1, weight_decay=1e-2)

# 2. Training Loop 내부에서 L1 규제 직접 추가
l1_norm = sum(p.abs().sum() for p in model.parameters())
loss = loss_fn(pred, y) + (1e-3 * l1_norm)

# 최종 Loss = MSE + L1_penalty + L2_penalty(Internal)
loss.backward(); optimizer.step()
```



R² Score

Evaluation Metric

역할 및 목적

모델의 성능 평가

"모델이 데이터를 얼마나 잘 설명하는가?"를 0~1 사이 점수로 해석

사용 시점

학습 완료 후 / 검증

모델의 최종 성적표 확인 시 사용



MSE

Loss Function

역할 및 목적

학습의 목표(Target)

예측 오차를 최소화하기 위해 미분 가능한 수치로 에러를 정량화

사용 시점

학습 루프 내부

Optimizer가 가중치를 업데이트할 때 기준 (Training)



Ridge / Lasso

Model Regularization

역할 및 목적

일반화 성능 개선

손실 함수에 패널티를 추가하여 과적합 (Overfitting)을 방지

사용 시점

모델 설계 / 손실 정의

학습이 너무 '잘' 되어버리는 것을 제어

